



THE UNIVERSITY OF
MELBOURNE

COMP90050 Advanced Database Systems

Semester 1, 2026

Lecturer: Farhana Choudhury (PhD)

Live lecture – Week 6





Today's topics

- Group project discussion
- Concurrency problem – continuation
 - Activities on dependencies and isolation concepts



Concurrency problem

Multiple concurrently running transactions may cause conflicts

- **Still we/DBMS need to allow concurrent runs as much as possible for a better performance, while avoiding conflicts**

What we need to know –

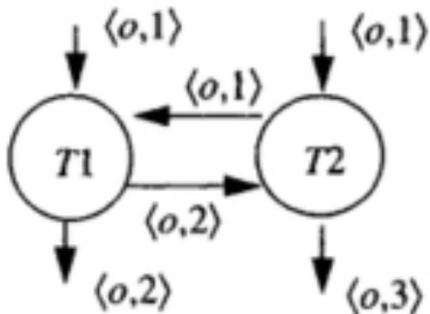
- What are the possible conflicts/dependencies
- Given a set of concurrent transactions, can we/DBMS determine whether there will be any conflict or not?
- Can conflicts be avoided (without making any change to the intended final output/final state of the database)?

Dependencies

When dependency graph has cycles then there is a violation of isolation and a possibility of inconsistency.

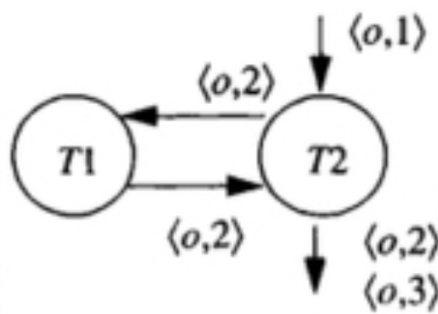
Lost Update

T2 READ $\langle o, 1 \rangle$
T1 WRITE $\langle o, 2 \rangle$
T2 WRITE $\langle o, 3 \rangle$



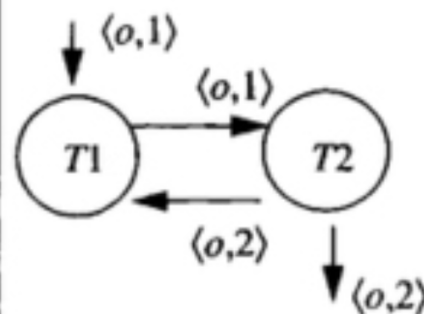
Dirty Read

T2 WRITE $\langle o, 2 \rangle$
T1 READ $\langle o, 2 \rangle$
T2 WRITE $\langle o, 3 \rangle$



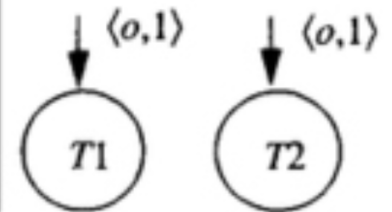
Unrepeatable Read

T1 READ $\langle o, 1 \rangle$
T2 WRITE $\langle o, 2 \rangle$
T1 READ $\langle o, 2 \rangle$

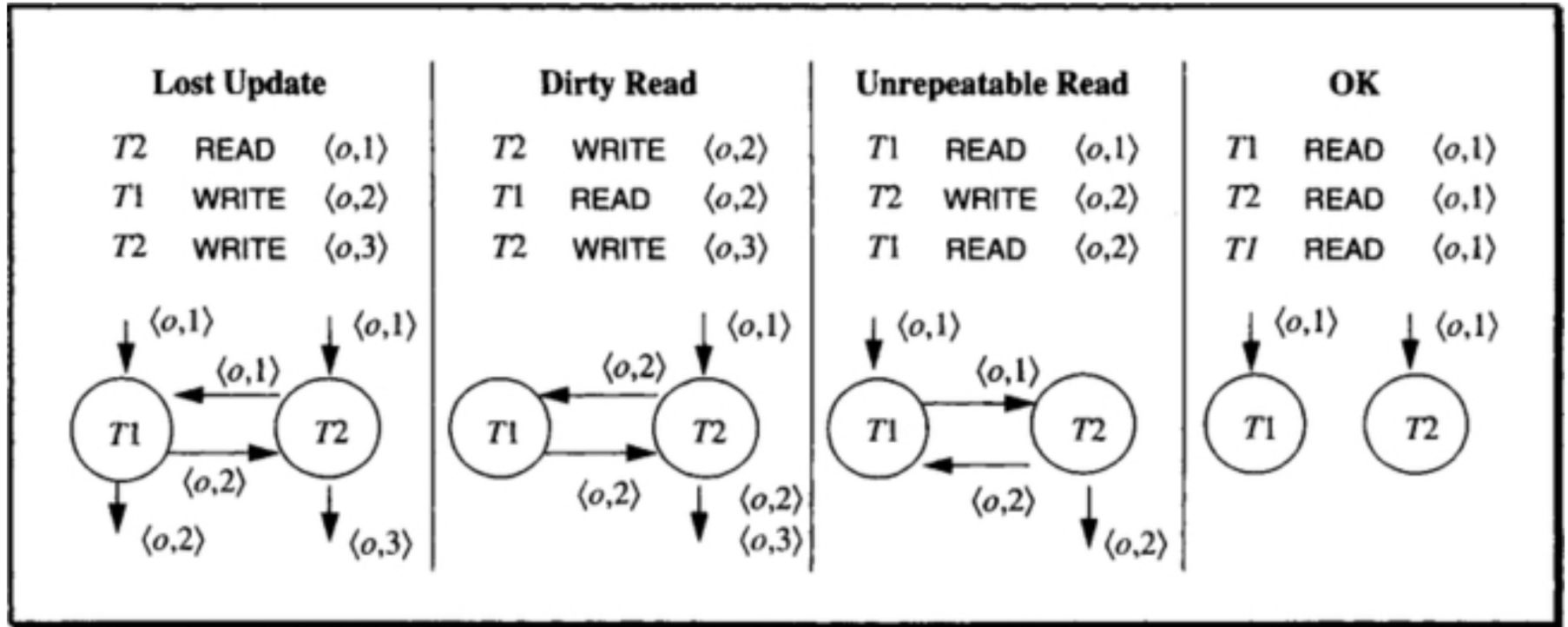


OK

T1 READ $\langle o, 1 \rangle$
T2 READ $\langle o, 1 \rangle$
T1 READ $\langle o, 1 \rangle$



Some activities !



Time for a poll - [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)





Dependency relations

Time for an activity!

$H1 = \langle (T1, R, O1), (T2, W, O5), (T1, W, O3), (T3, W, O1), (T5, R, O3), (T3, W, O2), (T5, R, O4), (T4, R, O2), (T6, W, O4) \rangle$

$DEP(H1) = \{ \langle T1, O1, T3 \rangle, \langle T1, O3, T5 \rangle, \langle T3, O2, T4 \rangle, \langle T5, O4, T6 \rangle \}$

Dependency relations

Time for an activity!

$H1 = \langle (T1, R, O1), (T2, W, O5), (T1, W, O3), (T3, W, O1), (T5, R, O3), (T3, W, O2), (T5, R, O4), (T4, R, O2), (T6, W, O4) \rangle$

$DEP(H1) = \{ \langle T1, O1, T3 \rangle, \langle T1, O3, T5 \rangle, \langle T3, O2, T4 \rangle, \langle T5, O4, T6 \rangle \}$

Given two different order of executions, can we have some insight on the final output/state of the database?

$H2 = \langle (T1, R, O1), (T3, W, O1), (T3, W, O2), (T4, R, O2), (T1, W, O3), (T2, W, O5), (T5, R, O3), (T5, R, O4), (T6, W, O4) \rangle$

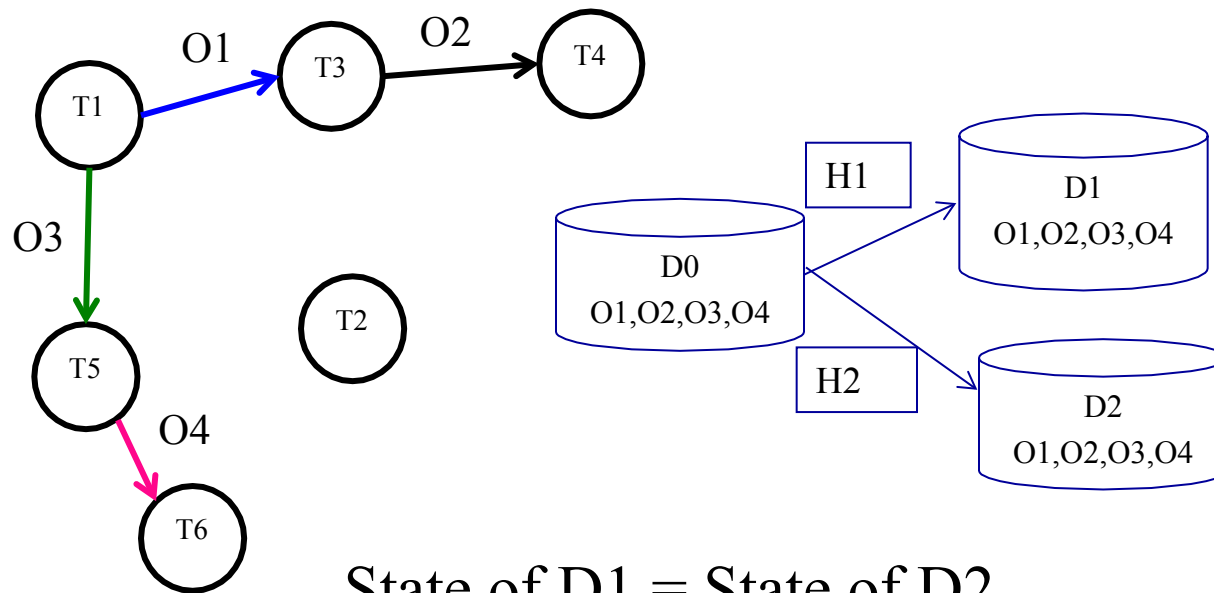
$DEP(H2) = \{ \langle T1, O1, T3 \rangle, \langle T1, O3, T5 \rangle, \langle T3, O2, T4 \rangle, \langle T5, O4, T6 \rangle \}$

$DEP(H1) = DEP(H2)$

Dependency relations - equivalence

$DEP(H1) = \{ \langle T1, O1, T3 \rangle, \langle T1, O3, T5 \rangle, \langle T3, O2, T4 \rangle, \langle T5, O4, T6 \rangle \}$

$DEP(H2) = \{ \langle T1, O1, T3 \rangle, \langle T1, O3, T5 \rangle, \langle T3, O2, T4 \rangle, \langle T5, O4, T6 \rangle \}$



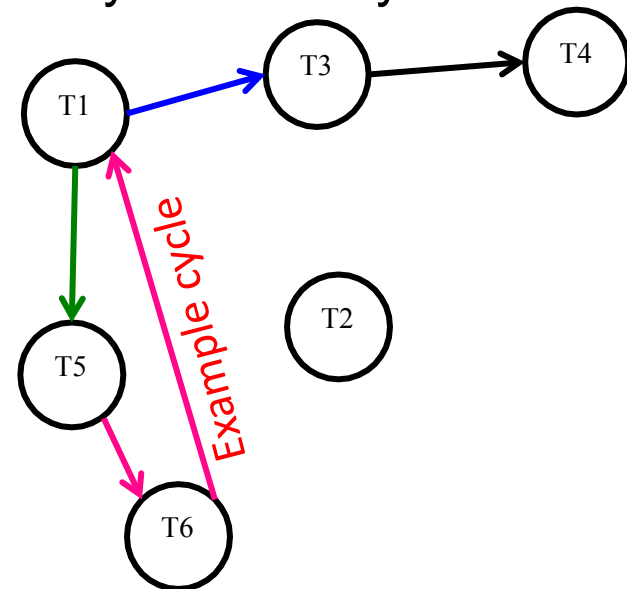
Goal: Can we run transactions concurrently, but still have the same final output/state of the database as if the transactions are serially executed?

Isolated history

Wormhole theorem: A history is isolated (that is, equivalent to a serial history) if and only if it has no wormholes.

A transaction T' is called a wormhole transaction if $T' \in \text{Before}(T) \cap \text{After}(T)$ (i.e., there will be a cycle in the dependency graph if there is a wormhole)

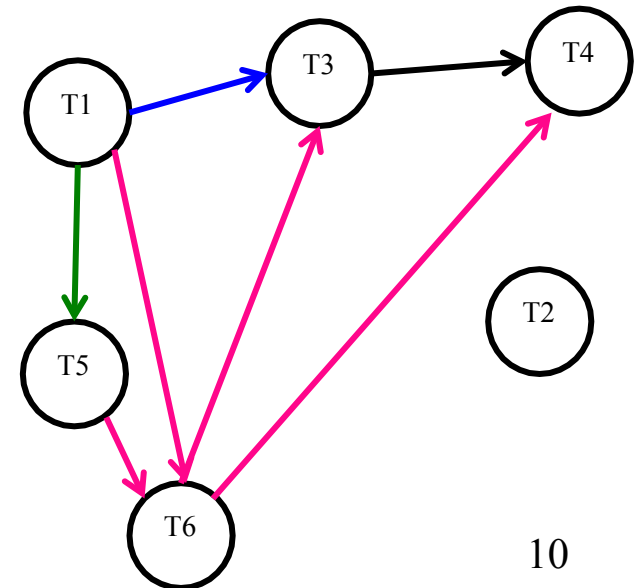
So, given a history, how can we determine whether it is 'isolated history' (that is, equivalent to a serial history)? – We try to find a cycle.



Isolated history

Activity - Give an example order of execution avoiding conflicts (without making any change to the intended final output/final state of the database)?

[Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)





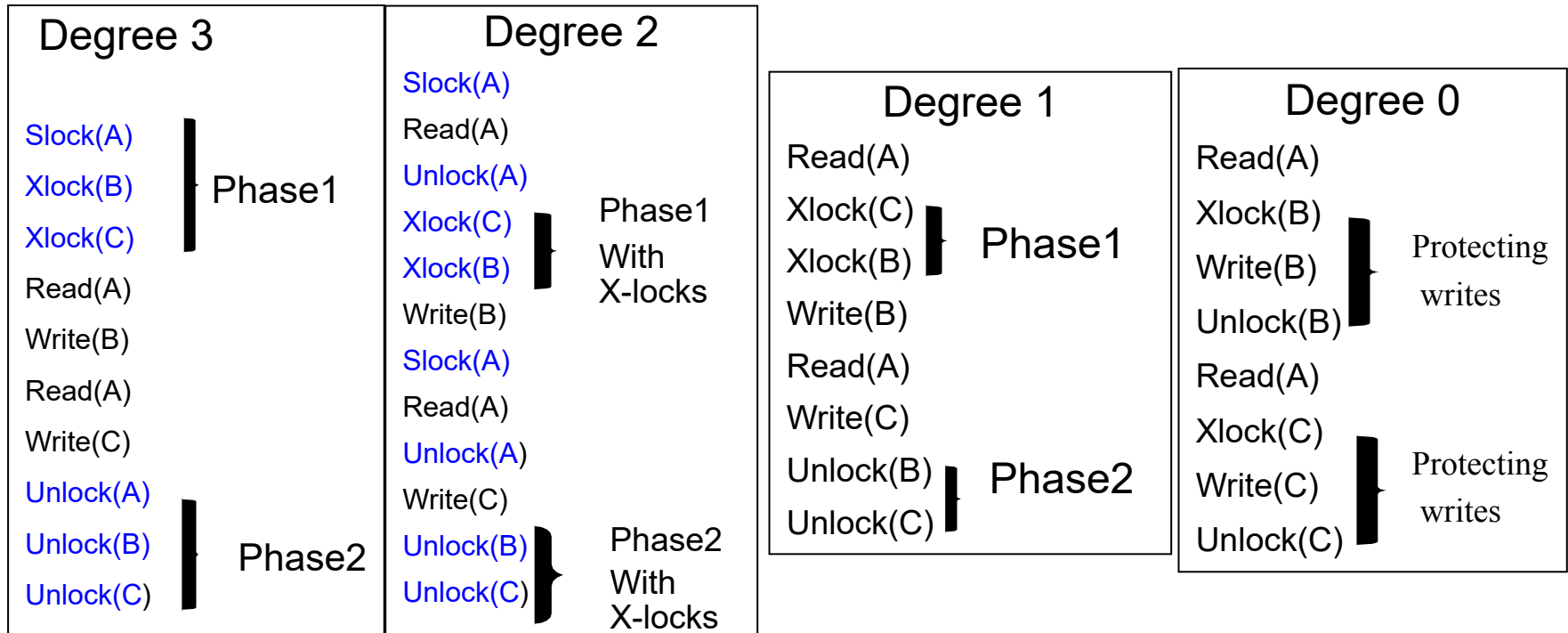
Concurrency problem

What we need to know –

- What are the possible conflicts/dependencies
- Given a set of concurrent transactions, can we/DBMS determine whether there will be any conflict or not?
- Can conflicts be avoided (without making any change to the intended final output/final state of the database)? - **Yes, locks – as that ensures there's no wormhole!**

Locking theorem: If all transactions are **well formed** (READ, WRITE and UNLOCK operation is covered earlier by a corresponding lock operation) and **two-phased** (locks are released only at the end), then any **legal** (does not grant conflicting grants) history will be isolated.

Degrees of Isolation



**More restrictive locks mean less/no conflict,
but the overall transaction throughput gets slower**



Isolation Concepts ...

In SQL2 one can declare isolation level as follows:

```
SET TRANSACTION ISOLATION LEVEL {READ UNCOMMITTED |  
READ COMMITTED | REPEATABLE READ | SERIALIZABLE}
```

Slight difference with the four degrees of isolation

- SERIALIZABLE – degree 3
- REPEATABLE READ – like degree 3, but other transactions can insert new rows
- READ COMMITTED – Degree 2*
- READ UNCOMMITTED – Degree 0

*Options can also be paired with SNAPSHOT on/off



Summary

Multiple concurrently running transactions may cause conflicts

- **Different types of conflicts**
- **Avoiding conflicts – using locks**

Later we will see –

- More types of locks
- Relaxed isolation – for better performance